

•



- [Feature Engineering](#)
- Best Practices for Plans

On this page

Best Practices for Plans

Was this page helpful? 

Within a map or metric operator, you cannot refer to a field that has been added inside that same operator. In case of such dependencies, you have to use more than one operator to calculate the final field. Also, performance considerations need to be taken into consideration when organizing your maps and metrics into plans.

The following sections describe best practices regarding maps, metrics and plans.

A simple plan

Due to performance reasons, it is recommended to organize map and metrics operators in a single plan, containing all feature engineering logic.

In a simplified example, the plan could have the following operators:

1. A map operator to calculate simple features.
2. A metrics operator to calculate time-based aggregations.
3. Typically another map operator to calculate features based on the metrics.

Large number of metrics

It can happen that the plan has to calculate hundreds of metrics.

Due to limitations of Java, when a metrics operator contains around 200 metrics, the plan might crash. This number depends on the length of the metrics' PQL code. The solution for this is to separate the metrics into two or more operators.

When separating the metrics into two or more different metrics operators, it's a good practice to keep metrics in the same operator if they have the same field in the first place in the *Group By* field. For example, if you have some metrics that group events by *Card*, and some other metrics that group events by *Card* and *ShippingAddress*, then you can keep these metrics in the same operator to speed up the computation.

This way, Spark has to do only one shuffle for those metrics, which is usually the most time-consuming part of Spark computations. However, this technique has the cost of higher memory usage.

Scheduled metrics

When you have scheduled metrics in a plan, Pulse has to recompute everything up to that point in the plan. If you have a lot of computations before the scheduled metrics operator, it might create a big overhead in computation time. This is usually a non-issue since scheduled metrics are normally computed during the night and are not subject to the real-time constraints. Nevertheless be aware of this in case you think your computations will be taking longer than a night in production.

As an exception to the one plan rule, having two plans in this situation can be a trade-off between storage space and computational times.

Addressing errors in plan definition

When defining the components of a **Plan**, you may encounter errors that would result in failures during the [plan execution](#).

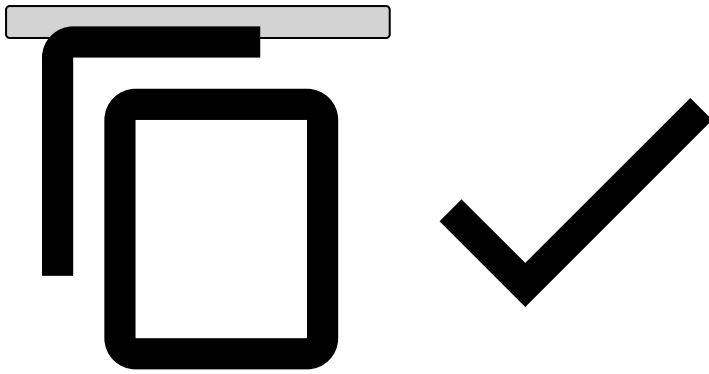
Some of those errors are detectable when defining the Plan, and Pulse will show warnings to allow them to be fixed before deploying the plan to production.

One such error is defining [fields](#) based on a [PQL expression](#) (e.g. within a [Metrics or Map operator](#)). Operators that require null coalescing (i.e. [default fallback values](#)) will produce a warning in the UI if the coalescing operator (`??`) and value are missing:

To fix this situation, provide a fallback value in the PQL expression input field, e.g. `?? 0` or `?? ""`.

A similar warning will be shown if built-in functions are used in collections (like `List` s, `Set` s or `Map` s). For example, an expression like

```
["test", "test1", "test2"].contains("testing")
```

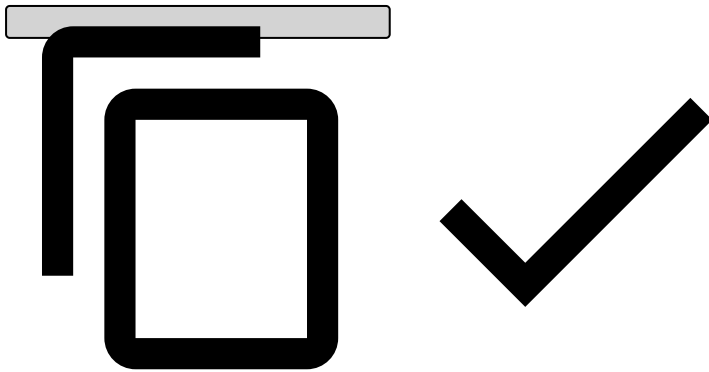


will fail because the `contains` function is not supported in collections used in the PQL fields of Map Operators or Metric Operators.

Attempting to use such functions in that context will result in a warning in the UI:

The alternative is to use [UDFs \(User Defined Functions\)](#), e.g.:

```
com.feedzai.pulse.pkernel.runtime.CollectionUDF.contains(["test", "test1", "test2"], "testing")
```



Besides these preventive warnings, Pulse will also allow recovering from execution errors that are not detectable at design time. Refer to [Automatic Application Repair](#) and [Workflow recovery](#) for more information on how to recover from execution errors.